Лекция 11: монада IO и взаимодействие с внешним миром

Функциональное программирование на Haskell

Алексей Романов

23 октября 2026 г.

МИЭТ

Монада IO

- Среди многих монад стандартной библиотеки, `IO` играет особую роль.
- Значение типа `IO a` описывает вычисление с результатом типа `a` и побочными эффектами.
- В первую очередь имеются в виду ввод-вывод, как видно из названия, но также изменяемые переменные и т.д.

Программы командной строки

- Исполняемая программа в Haskell задаётся значением `main :: IO a` (почти всегда `IO ()`).
- Это аналог `int main(int argc, char** argv)` в C, `public static void main(String[] args)` в Java и т.д.
- Соответственно, есть доступ к аргументам командной строки и возможность указать код выхода:
- `System.Environment.getArgs :: IO [String]`. Там же есть функции для чтения переменных окружения.
- `System.Exit.exitWith :: ExitCode -> IO a`.

Интерактивные программы

- Из консоли можно читать текст, вводимый пользователем:
  ```
  getLine ::
  ```

Интерактивные программы

- Из консоли можно читать текст, вводимый пользователем:
  ```
  getLine :: IO String
  ```
- И писать в неё:
  ```
  putStr, putStrLn ::
  ```

Интерактивные программы

- Из консоли можно читать текст, вводимый пользователем:
  ```
  getLine :: IO String
  ```
- И писать в неё:
  ```
  putStr, putStrLn :: String -> IO ()
  print :: Show a => a -> IO ()
  ```

Интерактивные программы

- Из консоли можно читать текст, вводимый пользователем:
  ```
  getLine :: IO String
  ```
- И писать в неё:
  ```
  putStr, putStrLn :: String -> IO ()
  print :: Show a => a -> IO ()
  ```
- Простой пример: читаем любую строку, выводим её длину и повторяем (а на `"quit"` выходим):

Интерактивные программы

- Из консоли можно читать текст, вводимый пользователем:
  ```
  getLine :: IO String
  ```
- И писать в неё:
  ```
  putStr, putStrLn :: String -> IO ()
  print :: Show a => a -> IO ()
  ```
- Простой пример: читаем любую строку, выводим её длину и повторяем (а на `"quit"` выходим):
  ```
  main = do
    line <- getLine
    if line /= "quit"
      then do
        print (length line)
        main
      else
        pure ()  -- или exitSuccess
  ```

Обход с IO: for и for_

- Те же обходы, но функция передаётся последней:

```
import Data.Traversable (for)
import Data.Foldable (for_)
for = flip traverse, for_ = flip traverse_.
```

- for собирает результаты; for_ отбрасывает их, сохраняя эффекты.

Обход с IO: for и for_

- Те же обходы, но функция передаётся последней:

```
import Data.Traversable (for)
import Data.Foldable (for_)
for = flip traverse, for_ = flip traverse_.
```

- for собирает результаты; for_ отбрасывает их, сохраняя эффекты.
- Так удобно записывать действие из нескольких строк:

```
for_ [1..3] $ \i -> do
  print i
  print (i * i)
```

Для каждого элемента печатаем число и его квадрат.

Условное выполнение: `when`

- Функция из упражнения в лекции 9:

```
import Control.Monad (when)

when :: Applicative f => Bool -> f () -> f ()
```

- При `True` выполняет действие, при `False` возвращает `pure ()` без его эффектов.

Условное выполнение: when

- Функция из упражнения в лекции 9:

```
import Control.Monad (when)

when :: Applicative f => Bool -> f () -> f ()
```

- При `True` выполняет действие, при `False` возвращает `pure ()` без его эффектов.
- Например, условный диагностический вывод:

```
report :: Bool -> IO ()
report debug = do
  when debug (putStrLn "Debug mode")
  putStrLn "Running"
```

`report False` печатает только `"Running"`.

Ловушки консоли: буферизация

- В скомпилированной программе `stdout` буферизуется построчно (а при перенаправлении в файл или канал — блоками). Поэтому

```
putStr "Введите имя: "
name <- getLine
```

не покажет приглашение, пока пользователь не нажмёт Enter.

Ловушки консоли: буферизация

- В скомпилированной программе `stdout` буферизуется построчно (а при перенаправлении в файл или канал — блоками). Поэтому

  ```
  putStr "Введите имя: "
  name <- getLine
  ```

  не покажет приглашение, пока пользователь не нажмёт Enter.
- Решения: `hFlush stdout` после `putStr` или `hSetBuffering stdout NoBuffering` в начале `main` (модуль `System.IO`).
- В GHCi и `stack repl` этого не видно: там буферизация отключена. Проверяйте `stack run`.

Ловушки консоли: кодировки

- Кодировка `stdin`/`stdout` и `readFile`/`writeFile` определяется окружением; UTF-8 не гарантирована.
- Под Windows с кодовой страницей 866 или 1251 вывод кириллицы может закончиться ошибкой вроде `<stdout>: hPutChar: invalid argument (invalid character)` или крякозябрами.

Ловушки консоли: кодировки

- Кодировка `stdin`/`stdout` и `readFile`/`writeFile` определяется окружением; UTF-8 не гарантирована.
- Под Windows с кодовой страницей 866 или 1251 вывод кириллицы может закончиться ошибкой вроде `<stdout>: hPutChar: invalid argument (invalid character)` или крякозябрами.
- Решение: `chcp 65001` перед запуском и/или в программе

```
hSetEncoding stdout utf8
hSetEncoding stdin utf8
```

- Для файлов — `hSetEncoding h utf8` после `openFile` (или `withFile`).

Изменяемые переменные: IORef

- Модуль Data.IORef:

```
newIORef :: a -> IO (IORef a)
readIORef :: IORef a -> IO a
writeIORef :: IORef a -> a -> IO ()
modifyIORef' :: IORef a -> (a -> a) -> IO ()
```

- Используйте modifyIORef' (строгий), а не modifyIORef: последний накапливает отложенные вычисления ровно как foldl в лекции 8.
- Для нескольких потоков нужны atomicModifyIORef', MVar или STM (лекция 13).

Исключения (кратко)

- Модуль `Control.Exception`:

```
throwIO :: Exception e => e -> IO a
catch   :: Exception e => IO a -> (e -> IO a) -> IO a
try     :: Exception e => IO a -> IO (Either e a)
evaluate :: a -> IO a
bracket :: IO a -> (a -> IO b) -> (a -> IO c) -> IO c
```

- Нужно определить, какие исключения ловить:

```
import System.IO (readFile')

try @IOException (readFile' path)
```

  Ловить всё (`SomeException`) — обычно плохая идея.

- `error` и неполные сопоставления тоже исключения (`ErrorCall`, `PatternMatchFail`), но лучше их не ловить, а не допускать.

Исключения и ленивость

- `pure` не вычисляет результат:

  ```
  try @ArithException (pure (div (1 :: Int) 0))
  ```

  Получим `Right` с отложенным вычислением; ошибка возникнет уже вне `try`.

Исключения и ленивость

- `pure` не вычисляет результат:

  ```
  try @ArithException (pure (div (1 :: Int) 0))
  ```

  Получим `Right` с отложенным вычислением; ошибка возникнет уже вне `try`.
- Чтобы вычислить результат внутри `try`:

  ```
  try @ArithException (evaluate (div (1 :: Int) 0))
  ```

  Здесь получим `Left DivideByZero`.
- `evaluate` вычисляет только до *WHNF* (лекция 8).
- Ошибки ленивого `readFile` тоже могут возникнуть вне `try`; `readFile'` читает файл сразу.

Работа с файлами

- Модуль `System.IO`.
- Напомню, `type FilePath = String`.
- Простейшие операции это чтение и запись в файл:

```
readFile :: FilePath -> IO String
writeFile, appendFile ::
  FilePath -> String -> IO ()
```

- Или можно открыть файл один раз, получить `Handle` (дескриптор файла) и работать с ним:

```
openFile, openBinaryFile ::
  FilePath -> IOMode -> IO Handle
hPutStrLn :: Handle -> String -> IO ()
hGetStr...
```

Работа с файлами

- Модуль `System.IO`.
- Напомню, `type FilePath = String`.
- Простейшие операции это чтение и запись в файл:

```
readFile :: FilePath -> IO String
writeFile, appendFile ::
  FilePath -> String -> IO ()
```

- Или можно открыть файл один раз, получить `Handle` (дескриптор файла) и работать с ним:

```
openFile, openBinaryFile ::
  FilePath -> IOMode -> IO Handle
hPutStrLn :: Handle -> String -> IO ()
hGetStr...
```

- Функции работы с консолью сводятся к этим:

```
getLine = hGetLine stdin
```

Работа с файлами (2)

- В чём проблема с кодом вроде

```
do
  file <- openFile path ReadWriteMode
  ...
  hClose file
```

Работа с файлами (2)

- В чём проблема с кодом вроде

```
do
  file <- openFile path ReadWriteMode
  ...
  hClose file
```

- Что, если на каком-то из шагов случится исключение? Мы остаёмся с открытым файлом.
- Если это случится много раз, программа вылетит.

Работа с файлами (2)

- В чём проблема с кодом вроде

```
do
  file <- openFile path ReadWriteMode
  ...
  hClose file
```

- Что, если на каком-то из шагов случится исключение? Мы остаёмся с открытым файлом.
- Если это случится много раз, программа вылетит.
- Чтобы закрыть файл и при исключении:

```
withFile path ReadWriteMode $ \file -> do
  ...
```

- Это частный случай функции `bracket`.

Ленивый ввод-вывод

- `hGetContents` возвращает строку: файл читается по мере её вычисления.
- С одной стороны, это хорошо: не нужно явно указывать, сколько читать.
- Но попытка дочитать строку после закрытия файла вызывает ошибку:

```
wrong = do
  fileData <-
    withFile "test.txt" ReadMode hGetContents
  putStr fileData
```

- Нужно использовать данные внутри `withFile`:

```
right = withFile "test.txt" ReadMode $ \file -> do
  fileData <- hGetContents file
  putStr fileData
```

Строгий ввод-вывод

- С GHC 9.0 в `System.IO` есть строгие варианты `readFile'`, `hGetContents'`, `getContents'`: они читают всё сразу, и проблем с закрытием файла у них нет.
- Обычно лучше использовать их или строгие варианты `Text`/`ByteString` со следующего слайда. Ленивые варианты `readFile` читают по требованию.
- Ленивый ввод-вывод остаётся полезным для потоковой обработки больших файлов, но для этого есть и более предсказуемые библиотеки (conduit, streamly).

`ByteString` и `Text`

- Мы знаем, что `String` занимает очень много места в памяти, что ухудшает и время работы.
- Вместо него есть `Data.ByteString.ByteString`, по сути представляющий собой массив байтов, и `Data.ByteString.Lazy.ByteString` как ленивый список таких массивов.
- Многие функции для работы с ними называются так же, как для `String`, но живут в соответствующих модулях (поэтому используется `import qualified`).
- Они годятся для бинарных данных (или ASCII, с помощью `Data.ByteString[.Lazy].Char8`).
- Для текста аналогичный пакет `text` и типы в модулях `Data.Text[.Lazy]`.

Текст и байты: UTF-8

- `ByteString` содержит байты, `Text` — текст. Для перехода между ними нужно выбрать кодировку.
- `Data.ByteString.Char8` *не декодирует UTF-8*: каждый байт интерпретируется как отдельный символ.

Текст и байты: UTF-8

- `ByteString` содержит байты, `Text` — текст. Для перехода между ними нужно выбрать кодировку.
- `Data.ByteString.Char8` *не декодирует UTF-8*: каждый байт интерпретируется как отдельный символ.
- Для UTF-8 есть модуль `Data.Text.Encoding`:

```
decodeUtf8' :: ByteString ->
  Either UnicodeException Text
encodeUtf8 :: Text -> ByteString
```

- `decodeUtf8'` возвращает `Left` при некорректной последовательности байтов; `Right` содержит декодированный текст.

Случайные значения

- Модуль `System.Random` (пакет `random`). Класс `RandomGen` описывает чистые генераторы псевдослучайных чисел. Например:

```
genWord64 :: RandomGen g => g -> (Word64, g)
```

- Получаем число и *новое состояние* генератора. `Word64` — 64-битное беззнаковое целое из `Data.Word`.
- Стандартный генератор `StdGen` имеет экземпляр `RandomGen`. Его можно создать из начального значения (*seed*):

```
mkStdGen :: Int -> StdGen
```

Случайные значения: тип результата

- `Uniform a`: равномерное распределение по всем значениям типа `a` (например, `Bool` или `Int`).
- `UniformRange a`: равномерное распределение в заданном диапазоне (например, для `Int`, `Integer` или `Double`).
- В `System.Random` есть чистые функции:

```
uniform :: (Uniform a, RandomGen g) =>
  g -> (a, g)
uniformR :: (UniformRange a, RandomGen g) =>
  (a, a) -> g -> (a, g)
```

- Результат и новое состояние генератора — знакомая схема `State`.

Случайные значения: воспроизводимость

- Передаём новое состояние в следующий вызов:

```
g0 = mkStdGen 42
(n1 :: Int, g1) = uniformR (1, 100) g0
(n2 :: Int, g2) = uniformR (1, 100) g1

ghci> (n1, n2)
(49,24)
```

- Если снова передать `g0`,

Случайные значения: воспроизводимость

- Передаём новое состояние в следующий вызов:

```
g0 = mkStdGen 42
(n1 :: Int, g1) = uniformR (1, 100) g0
(n2 :: Int, g2) = uniformR (1, 100) g1

ghci> (n1, n2)
(49,24)
```

- Если снова передать `g0`, получим `(49,49)`.
- При той же версии `random` повторный запуск с тем же seed даёт ту же последовательность.

Случайные значения в IO

- В `System.Random.Stateful` генераторы сами хранят изменяемое состояние:

```
uniformM :: (Uniform a, StatefulGen g m) =>
  g -> m a
uniformRM :: (UniformRange a, StatefulGen g m) =>
  (a, a) -> g -> m a
```

  `StatefulGen g m` связывает генератор `g` с монадой `m`.

- Глобальный генератор обновляется атомарно:

```
globalStdGen :: AtomicGenM StdGen
```

- Например, бросок кубика:

```
rollDie :: IO Int
rollDie = uniformRM (1, 6) globalStdGen
```

  Каждый вызов обновляет состояние генератора.

Из чистого генератора в IO

- `AtomicGenM` — обёртка над `IORef`:

```
newtype AtomicGenM g = AtomicGenM (IORef g)
atomicModifyIORef' :: IORef g -> (g -> (g, a)) -> IO a
```

- Реализуем библиотечную функцию (тип для `IO`):

```
applyAtomicGen :: (g -> (a, g)) -> AtomicGenM g -> IO a
applyAtomicGen step (AtomicGenM ref) = ???
```

Из чистого генератора в IO

- `AtomicGenM` — обёртка над `IORef`:

```
newtype AtomicGenM g = AtomicGenM (IORef g)
atomicModifyIORef' :: IORef g -> (g -> (g, a)) -> IO a
```

- Реализуем библиотечную функцию (тип для `IO`):

```
applyAtomicGen :: (g -> (a, g)) -> AtomicGenM g -> IO a
applyAtomicGen step (AtomicGenM ref) =
  atomicModifyIORef' ref $ \g ->
    let (x, g') = step g
    in (g', x)
```

- Тот же бросок кубика через `applyAtomicGen`:

```
rollDie :: IO Int
rollDie = applyAtomicGen (uniformR (1, 6)) globalStdGen
```

Побег из `IO`

- Если у нас есть значение типа `IO a`, можно ли превратить его просто в `a`?

Побег из `IO`

- Если у нас есть значение типа `IO a`, можно ли превратить его просто в `a`?
- В обычном коде нет.

Побег из `IO`

- Если у нас есть значение типа `IO a`, можно ли превратить его просто в `a`?
- В обычном коде нет. Но есть функция `unsafePerformIO :: IO a -> a`.
- Но само название говорит о её небезопасности.
- Конкретное использование может быть и безопасным, но для этого надо знать довольно много о внутренностях Haskell.

Побег из `IO`

- Если у нас есть значение типа `IO a`, можно ли превратить его просто в `a`?
- В обычном коде нет. Но есть функция `unsafePerformIO :: IO a -> a`.
- Но само название говорит о её небезопасности.
- Конкретное использование может быть и безопасным, но для этого надо знать довольно много о внутренностях Haskell.
- Поэтому в рамках этого курса мы её использовать не будем, как и другие `unsafe*IO`.
- А есть ещё `accursedUnutterablePerformIO`...